

JAVA CASE STUDY
SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE
AWARD OF THE DEGREE OF
BACHELOR OF TECHNOLOGY
(Information Technology)



SUBMITTED BY:
NAME: Jasdeep Singh (2435556)

SUBMITTED TO:
Prof. Jaspreet Kaur

DEPARTMENT OF INFORMATION TECHNOLOGY
GURU NANAK DEV ENGINEERING COLLEGE LUDHIANA
(An Autonomous College Under UGC ACT)

CASE STUDY ON ENCAPSULATION IN JAVA

Problem Statement

In object-oriented programming, managing and protecting data is one of the most important challenges. In real-world applications, directly allowing access to data members of a class can lead to security issues, data inconsistency, and lack of control over how the data is modified.

For example, in a banking system, a user's account balance should not be directly accessible or modifiable by external classes. If unrestricted access is allowed, it may lead to invalid operations such as setting a negative balance or unauthorized data manipulation. Similarly, in student management systems or employee records, sensitive information must be protected from direct access.

To solve this problem, Java provides the concept of **Encapsulation**, which is the mechanism of wrapping data (variables) and code (methods) into a single unit (class) and restricting direct access to some of the object's components. This is achieved using access modifiers (private, public, protected) and getter-setter methods.

Encapsulation ensures data hiding, controlled access, and improved security of the application. It also enhances code maintainability and flexibility by allowing changes in implementation without affecting other parts of the program.

The purpose of this case study is to understand how encapsulation works in Java by designing classes with private data members, implementing getter and setter methods, and analyzing how it helps in building secure and well-structured applications.

Objectives

The main objectives of this case study are:

- To understand the concept of **Encapsulation** in Java.
- To study how data hiding is achieved using **private access modifiers**.
- To learn the use of **getter and setter methods** for controlled access to data.
- To understand the importance of protecting sensitive data in real-world

applications.

- To implement encapsulation using Java classes and objects.
- To improve programming skills by designing secure and structured code.
- To understand how encapsulation helps in **maintaining data integrity**.
- To study the role of encapsulation in building **flexible and maintainable software systems**.
- To analyze real-life applications where encapsulation is used (e.g., banking, student management systems).

System Design

Overview

The system is designed to demonstrate the concept of **Encapsulation** in Java. It focuses on protecting data by restricting direct access to class variables and allowing access only through well-defined methods.

In this system, a class contains **private data members** and **public getter and setter methods**. The private variables cannot be accessed directly from outside the class, ensuring data security. The getter methods are used to retrieve values, while setter methods are used to update values with proper validation.

The system works by creating an object of the class and interacting with its data through these methods, ensuring controlled and secure data handling.

Components of the System

Encapsulated Class (Main Class)

- Contains private data members
- Provides public getter and setter methods
- Ensures data hiding and controlled access

User / Calling Class (Driver Class)

- Creates object of the encapsulated class
- Accesses data using getter and setter methods

Structure of the System

Student → Encapsulated Class

↓

Main → Driver Class

Concepts Used

Encapsulation

Encapsulation is the process of **wrapping data (variables) and methods (functions)** into a single unit called a class. It restricts direct access to the data and allows it only through controlled methods.

In Java, encapsulation is achieved by declaring variables as **private** and providing **public getter and setter methods** to access and modify them.

Syntax Example:

```
class Student {  
    private String name; // private data member  
  
    public String getName() { // getter method  
        return name;  
    }  
  
    public void setName(String name) { // setter method  
        this.name = name;  
    }  
}
```

Data Hiding

Data hiding is a key feature of encapsulation. It ensures that sensitive data is **hidden from outside access** and can only be accessed through authorized methods.

- Achieved using **private access modifier**
- Prevents unauthorized changes
- Protects data integrity

Access Modifiers

Access modifiers define the visibility of variables and methods in a class.

- **private** → Accessible only within the class
- **public** → Accessible from anywhere
- **protected** → Accessible within package and subclasses

Encapsulation mainly uses **private variables + public methods**.

Getter Methods

Getter methods are used to **retrieve (read)** the value of private variables.

```
public int getAge() {  
    return age;  
}
```

Setter Methods

Setter methods are used to **modify (update)** the value of private variables, often with validation.

```
public void setAge(int age) {  
    if(age > 0) {  
        this.age = age;  
    }  
}
```

Advantages of Encapsulation

- Improves **data security**
- Provides **controlled access**
- Enhances **code maintainability**
- Supports **flexibility and reusability**
- Prevents **unauthorized data modification**

Implementation

Java Program

```
class Student {  
    // Private data members (data hiding)  
    private String name;  
    private int age;  
  
    // Getter method for name  
    public String getName() {  
        return name;  
    }  
  
    // Setter method for name  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

```
}

// Getter method for age
public int getAge() {
    return age;
}

// Setter method for age with validation
public void setAge(int age) {
    if (age > 0) {
        this.age = age;
    } else {
        System.out.println("Invalid age!");
    }
}
}

// Main class (Driver Class)
public class EncapsulationDemo {
    public static void main(String[] args) {

        // Creating object of Student class
        Student s = new Student();

        // Setting values using setter methods
        s.setName("Jasdeep");
        s.setAge(20);

        // Getting values using getter methods
        System.out.println("Name: " + s.getName());
        System.out.println("Age: " + s.getAge());
    }
}
```

Output

```
Name: Jasdeep
Age: 20
```

Procedure

Step 1: Open any Java IDE such as Eclipse, IntelliJ IDEA, or BlueJ.

Step 2: Create a new Java file named:

`EncapsulationDemo.java`

Step 3: Write the above Java program demonstrating encapsulation.

Step 4: Save the file.

Step 5: Compile the program using:

```
javac EncapsulationDemo.java
```

Step 6: Run the program using:

```
java EncapsulationDemo
```

Step 7: Observe the output displayed on the screen.

Advantages

- **Improves Data Security**
Sensitive data is hidden from direct access, reducing the risk of unauthorized modification.
- **Provides Controlled Access**
Data can only be accessed or modified through getter and setter methods, allowing validation and control.
- **Maintains Data Integrity**
Ensures that only valid data is stored in variables by applying conditions inside setter methods.
- **Enhances Code Maintainability**
Changes in implementation can be made without affecting other parts of the program.
- **Supports Flexibility**
Internal implementation can be changed without changing how other classes use the code.
- **Promotes Reusability**
Encapsulated classes can be reused in different parts of the application.
- **Reduces Complexity**
By hiding unnecessary details, it makes the program easier to understand and use.
- **Improves Modularity**
Code is organized into smaller, manageable units (classes), making development efficient.
- **Prevents Unauthorized Access**
Direct access to variables is restricted, ensuring better control over data usage.

Real Life Applications

Banking Systems

- Account details such as balance, PIN, and transaction history are kept private.
- Access is provided only through methods like deposit, withdraw, and check balance.
- Prevents unauthorized access and ensures secure transactions.

Student Management Systems

- Student data such as marks, grades, and personal details are hidden.
- Access is controlled using getter and setter methods.
- Ensures accurate and consistent student records.

Healthcare Applications

- Patient information such as medical history and reports is kept confidential.
- Only authorized methods allow viewing or updating data.
- Helps maintain privacy and data security.

Online Shopping Applications

- User details such as address, payment information, and order history are protected.
- Encapsulation ensures safe handling of sensitive data.
- Improves system reliability and user trust.

Vehicle Management Systems

- Vehicle properties such as speed, fuel level, and engine status are controlled internally.
- Methods ensure values remain within valid limits.

Game Development

- Player attributes such as health, score, and level are encapsulated.
- Prevents illegal modifications and maintains fair gameplay.

Mobile Applications

- User data and application settings are encapsulated within classes.
- Ensures secure and smooth functioning of applications.

Educational Software

- Different roles such as student, teacher, and administrator have controlled access to data.
- Prevents misuse of information and maintains system integrity.

Conclusion

Encapsulation is one of the fundamental concepts of object-oriented programming in Java. It allows developers to protect data by restricting direct access and providing controlled access through methods. This ensures that sensitive information remains secure and cannot be modified without proper validation.

In this case study, the concept of encapsulation was successfully studied and implemented using Java programming. The use of private data members along with getter and setter methods demonstrated how data hiding and controlled access can be achieved effectively.

The study also highlighted that encapsulation improves code maintainability, flexibility, and reusability. By separating the internal implementation from external usage, it becomes easier to modify and extend the program without affecting other parts of the system.

Encapsulation is widely used in real-world applications such as banking systems, healthcare applications, student management systems, and online platforms where data security is essential. It plays a crucial role in developing robust, secure, and well-structured software.

Overall, this case study helped in understanding the practical implementation, importance, advantages, and real-life applications of encapsulation in Java. It is an essential concept for building reliable and maintainable software systems.